

# **COL1000: Introduction to Programming**

## **Lectures 20-21**

**Keerti, Naveen, Madhukar, Venkata  
Department of CSE, IIT Delhi**

# Selection Sort

# Selection Sort

**Problem:** Sorting a list  $L$  storing  $n$  distinct integers.

```
For  $i = 0$  to  $n-1$ :
```

```
    # Invariant (pre):  $L[0:i]$  contains smallest  $i$  elements of  $L$  in increasing order
```

```
        Find index  $p$  of smallest element in sub-list  $L[i:]$ 
```

```
    and swap  $L[p], L[i]$ 
```

```
    # Invariant (post):  $L[0:i+1]$  contains smallest  $i+1$  elements of  $L$  in increasing order
```

SelectionSort(  $L$  )

# Selection Sort

**Problem:** Sorting a list L storing n distinct integers.

```
def selection_sort(L):
    n = len(L)

    for i in range(n-1):
        min = i

        # Find index of the smallest element in L[i:]
        for j in range(i+1, n):
            if L[j] < L[min]:
                min = j

        # Swap
        L[i], L[min] = L[min], L[i]
```

SelectionSort( L )

**Number of primitive operations:** At most  $c \cdot n$ <sup>2</sup>, for some constant c

# Bubble Sort

# Bubble Sort

**Problem:** Sorting a list  $L$  storing  $n$  distinct integers.

For  $i = 0$  to  $n-1$ :

# Invariant (pre):  $L[n-i:]$  contains largest  $i$  elements of  $L$  in increasing order

For each  $j$  from  $0$  to  $n-1$ :

Compare  $L[j]$  and  $L[j+1]$  and swap if former is larger

# Invariant (post):  $L[n-(i+1):]$  contains largest  $i+1$  elements of  $L$  in increasing order

BubbleSort(  $L$  )

# Bubble Sort

**Problem:** Sorting a list L storing n distinct integers.

```
def bubblesort(L):  
  
    n = len(L)  
  
    for i in range(n):  
  
        for j in range(n-i-1):  
  
            if(L[j] > L[j+1]): L[j],L[j+1] = L[j+1],L[j]
```

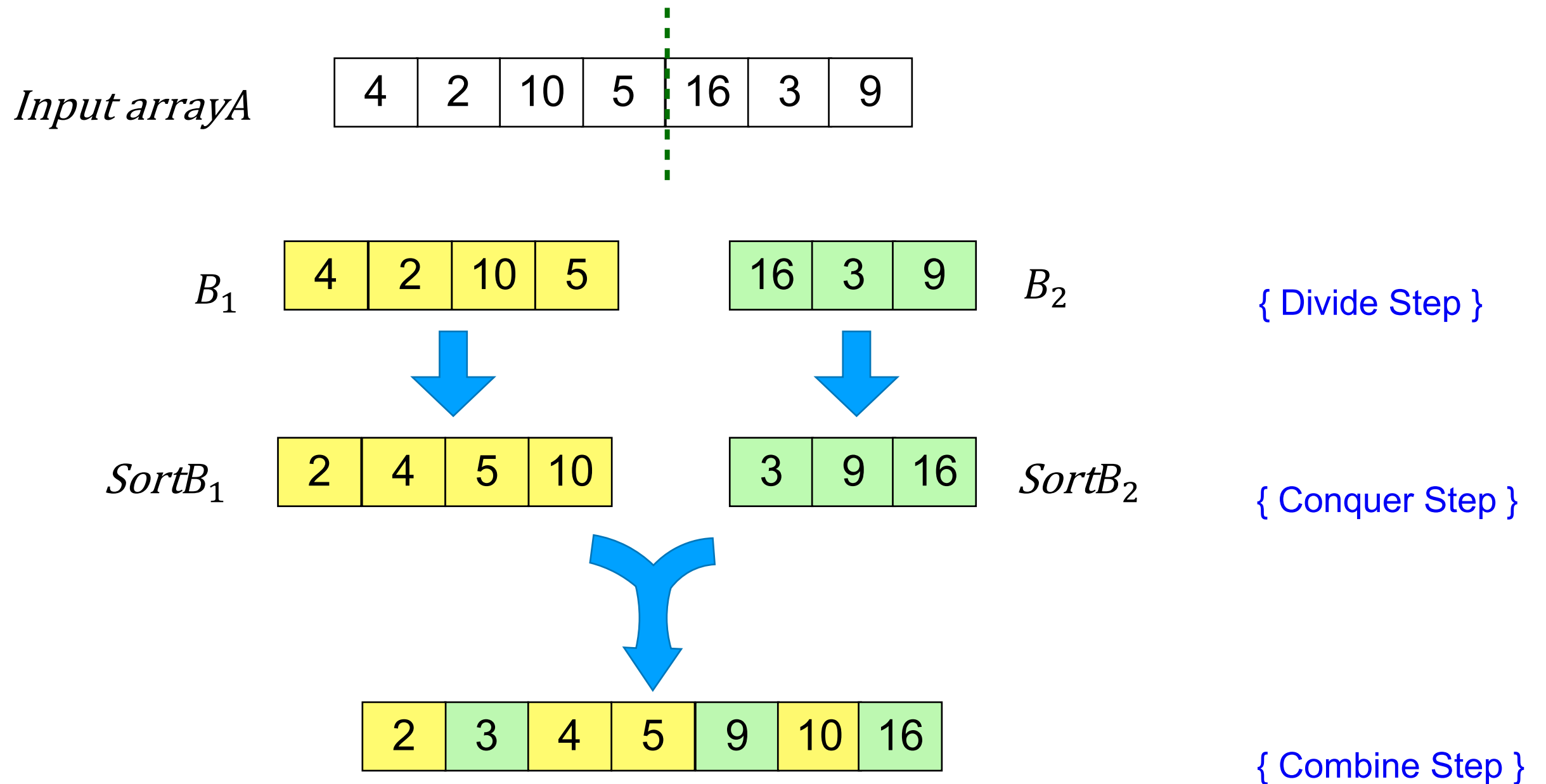
BubbleSort( L )

**Number of primitive operations:** At most  $c \cdot n^2$ , for some constant c

# Merge Sort

# Merge Sort

*A sorting algorithm based on divide-and-conquer approach:*



# Merging two sorted Lists

**Problem:** Merging two sorted lists into a new list.

```
def merge(A, B):  
  
    result = []  
    i = j = 0  
  
    while i < len(A) and j < len(B):  
        if A[i] <= B[j]:  
            result.append(A[i])  
            i = i+1  
        else:  
            result.append(B[j])  
            j = j+1  
  
    result.extend(A[i:]) # Copy remaining A  
    result.extend(B[j:]) # Copy remaining B  
    return result
```

Merge( A, B)

**Number of primitive operations:** At most  $c \cdot n$ , for some constant  $c$

# Merge Sort

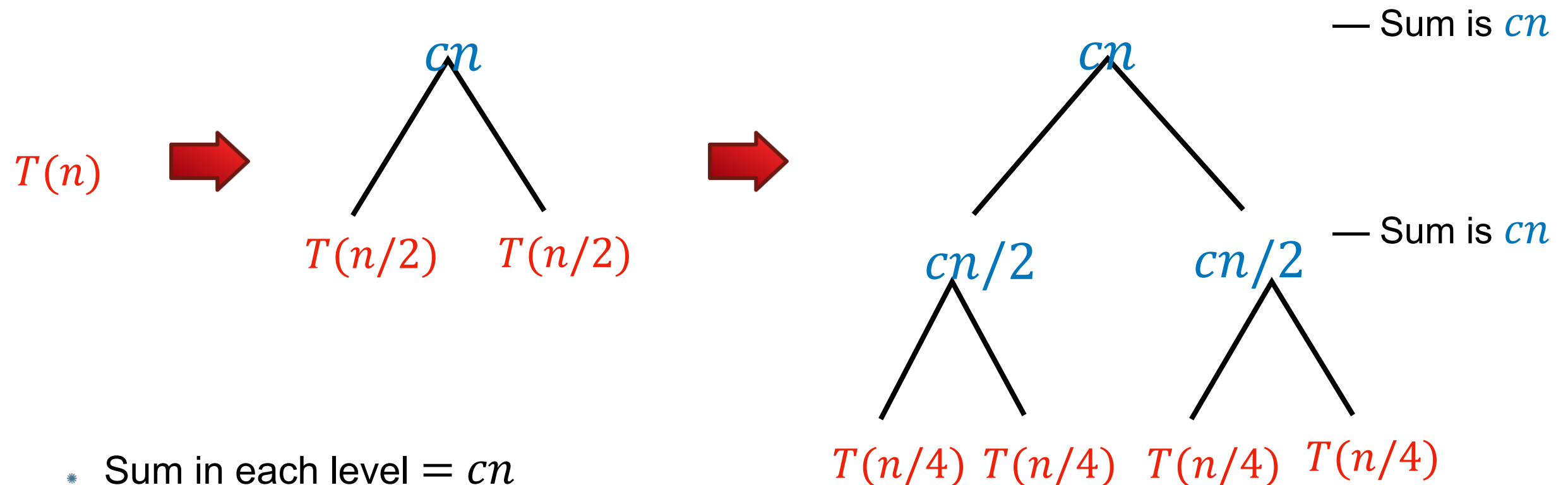
**Problem:** Sorting a list L storing n distinct integers.

```
def merge_sort(L):  
  
    if len(L) <= 1:  
        return L[:] # Copy base case  
  
    mid = len(L) // 2  
    A = merge_sort(L[:mid]) # Copy & recurse  
    B = merge_sort(L[mid:]) # Copy & recurse  
  
    return merge(A, B) # Merge returns new list  
  
L = [6, 4, 25, 2, 1]  
print(merge_sort(L))
```

MergeSort( L )

# Merge Sort Analysis

Let  $T(n)$  be bound on atomic steps taken by algorithm. Then,  $T(n) \leq 2T(\frac{n}{2}) + cn$



- \* Sum in each level =  $cn$
- \* Height =  $\log_2 n$
- \* So,  $T(n) = \text{At most } cn \log n$